
Machine Learning-Driven Variance Reduction in Monte Carlo Pricing

Stratified Clustering and Neural Network Control Variates

Benjamin Sulpice & Enzo Thuilliez

Supervised by **Professor Arnaud DUFAYS**

May 2026

Abstract

Background and Context. Derivative pricing and market risk quantification occupy a central position at the intersection of stochastic analysis, probability theory, and computational mathematics. Since Black, Scholes, and Merton formalised the risk-neutral pricing framework in 1973, Monte Carlo simulation has emerged as the method of choice for evaluating expectations under the risk-neutral measure $\mathbb{Q}$, owing to its dimension-independence and natural handling of path-dependent payoffs.

Core Problem Statement. The canonical Monte Carlo estimator converges at the slow rate $\mathcal{O}(N^{-1/2})$, where N denotes the number of simulated paths. Halving the statistical error requires quadrupling the computational budget—an unacceptable trade-off in production environments where prices and risk sensitivities must be computed in near-real time. Classical variance reduction techniques (antithetic variates, control variates, importance sampling) partially address this inefficiency but require problem-specific design that limits their applicability across diverse payoff structures.

Methodology. This memoir develops a unified machine learning framework for Monte Carlo variance reduction. Part I establishes the mathematical foundations: risk-neutral pricing under the Black–Scholes and Heston models and the theoretical basis of classical variance reduction. Part II introduces trajectory clustering (K-means and Gaussian Mixture Models) to partition the simulation space into homogeneous regimes, then constructs data-driven control variates using Random Forests and feedforward neural networks trained to approximate the conditional payoff expectation $\mathbb{E}^{\mathbb{Q}}[\Phi \mid \mathbf{x}]$.

Primary Findings. Clustering decomposes total payoff variance into between-cluster and within-cluster components, attacking both through stratification and local control variates respectively. Random Forest control variates achieve variance reduction factors of 10–50 $\times$ for standard European, Asian, and barrier options while requiring no domain expertise in control selection. Neural network control variates extend these gains to high-dimensional basket options and stochastic volatility settings, achieving correlations $\rho > 0.90$ with the payoff on pilot samples of $N_0 \geq 5 \times 10^4$ paths.

Key Implications. These results establish that learned variance reduction schemes match or exceed classical hand-crafted methods at a fraction of the design effort, with direct applicability to front-office pricing engines handling diverse exotic payoffs. The clustering-plus-local-learner pipeline generalises seamlessly to new products without analytical rederivation, positioning machine learning as a practical and principled component of production Monte Carlo infrastructure.

Contents

Abstract	1
Introduction	5
I Mathematical Foundations	7
1 Derivative Pricing and Monte Carlo Simulation	8
1.1 The Risk-Neutral Pricing Framework	8
1.1.1 Probability spaces and filtrations	8
1.1.2 No-arbitrage and the risk-neutral measure	8
1.1.3 The fundamental pricing equation	9
1.2 The Black–Scholes Model	9
1.2.1 Geometric Brownian motion	9
1.2.2 The Black–Scholes formula	9
1.3 The Heston Stochastic Volatility Model	10
1.3.1 Model specification	10
1.3.2 Euler–Maruyama discretisation	10
1.4 Monte Carlo Simulation for Derivative Pricing	11
1.4.1 The Monte Carlo estimator	11
1.4.2 Convergence and the $\mathcal{O}(N^{-1/2})$ rate	12
1.4.3 The variance problem and its consequences	13
1.4.4 Confidence intervals and practical diagnostics	13
1.5 Summary	14
2 Classical Variance Reduction Techniques	15
2.1 Antithetic Variates	15
2.2 Control Variates	16
2.3 Importance Sampling	16
2.4 Stratified Sampling	17
2.5 Summary and Motivation for Machine Learning	18

II	Machine Learning for Variance Reduction	19
3	Clustering of Monte Carlo Trajectories	20
3.1	From Paths to Feature Vectors	20
3.1.1	Why not cluster paths directly?	20
3.1.2	Financially motivated features	21
3.1.3	Feature normalisation	21
3.2	K-Means Clustering	22
3.2.1	The core idea	22
3.2.2	Objective and algorithm	22
3.2.3	Limitations of K-means	22
3.2.4	Choosing the number of clusters K	23
3.3	Gaussian Mixture Models	23
3.3.1	Model definition	23
3.3.2	The EM algorithm	23
3.3.3	Choosing K : BIC and AIC	24
3.4	Adaptive Simulation Allocation	24
3.5	Local Control Variates and Variance Reduction	25
3.5.1	Local control variates	25
3.5.2	Why clustering amplifies variance reduction	25
3.6	Summary	25
4	Random Forests for Payoff Approximation and Control Variates	26
4.1	Decision Trees	26
4.1.1	Formal definition	26
4.1.2	Illustrative example	26
4.2	Random Forests	27
4.3	Random Forest Control Variates	28
4.4	Comparison with Alternative Approaches	28
4.5	Summary	28
5	Neural Networks for Variance Reduction	30
5.1	From a Single Neuron to a Deep Network	30
5.1.1	The artificial neuron	30
5.1.2	The feedforward network	30
5.2	Universal Approximation	31
5.3	Training Neural Networks	31
5.3.1	Loss functions	31
5.3.2	Backpropagation	31
5.3.3	Adam optimiser	32

5.4	Neural Network Control Variates	32
5.4.1	The variance minimisation objective	32
5.4.2	Architecture	32
5.4.3	The training and pricing pipeline	33
5.5	Application to High-Dimensional Finance	33
5.6	Practical Limitations	33
5.7	Summary	33
Appendices		36
A Numerical Figures		36
A.1	Figure 1: Simulated Asset Trajectories	36
A.2	Figure 2: Empirical RMSE Convergence Rates	37
A.3	Figure 3: K-Means Cluster Structure in Feature Space	38
A.4	Figure 4: Neural Network Variance-Loss Trajectory	39
B Python Code Extracts		41
B.1	K-Means Clustering and Neyman Allocation (Step 3)	41
B.2	Local Random Forest Control Variate (Step 4)	44
B.3	Neural Network Variance-Loss Optimiser (Step 5)	47
C Supplementary Parameter Tables		52
Bibliography		54

Introduction

The valuation of financial derivatives and the quantification of market risk sit at the intersection of probability theory, stochastic analysis, and numerical mathematics. Over the past four decades, the rapid growth of derivative markets has driven a remarkable convergence between theoretical finance and applied mathematics, producing frameworks that now underpin the daily operations of trading desks, risk management units, and asset management firms worldwide.

At the core of modern derivative pricing lies a fundamental insight first formalised by Black, Scholes, and Merton in 1973: under certain no-arbitrage conditions, the price of a contingent claim can be expressed as a discounted expectation under a *risk-neutral* probability measure. For a broad class of payoff structures and underlying dynamics, this expectation does not admit a closed-form solution, and one must resort to numerical methods. Among these, **Monte Carlo simulation** occupies a privileged position: it is *dimension-agnostic*, naturally handles path-dependent payoffs, and extends seamlessly to multi-asset or multi-factor settings that defeat finite-difference or lattice-based approaches.

The central drawback of Monte Carlo methods is their *slow convergence*: the statistical error in a Monte Carlo estimator decreases only at rate $\mathcal{O}(N^{-1/2})$, where N is the number of simulated paths. Halving the error requires *quadrupling* the computational budget, an unacceptable trade-off when prices and Greeks must be computed in near-real time for hedging or client-quoting purposes. Variance reduction techniques seek to accelerate this convergence without sacrificing the generality that makes Monte Carlo attractive in the first place.

Classical variance reduction techniques such as antithetic variates, control variates, and importance sampling require significant domain expertise to be effective: the control must be chosen by hand, the change of measure must be derived analytically, and optimal parameters must be recomputed for each new product or model. This lack of adaptability limits their applicability in production environments handling hundreds of distinct payoff structures. Machine learning offers a fundamentally different approach: rather than designing variance reduction schemes by hand, one can learn them directly from simulated data, using flexible function approximators that adapt automatically to the payoff and model at hand, while preserving the martingale and no-arbitrage constraints required for consistent pricing.

This memoir is organised around the following progression. Part I establishes the mathematical foundations: the risk-neutral pricing framework (Chapter 1), and the theory of variance reduction in its classical forms (Chapter 2). Part II develops the

machine learning reformulation, showing how optimal control variates and importance sampling schemes can be *learned* from simulated data while preserving martingale and no-arbitrage constraints. Part III provides a numerical study, implementing and benchmarking the proposed methods on European, Asian, and barrier options under both Black–Scholes and Heston dynamics.

Throughout the memoir, we privilege rigour without sacrificing economic intuition. Each mathematical development is immediately contextualised by its practical implications for front-office pricing, hedging, and risk management.

Part I

Mathematical Foundations

Chapter 1

Derivative Pricing and Monte Carlo Simulation

1.1 The Risk-Neutral Pricing Framework

1.1.1 Probability spaces and filtrations

We work on a filtered probability space $(\Omega, \mathcal{F}, (\mathcal{F}_t)_{t \in [0, T]}, \mathbb{P})$, where $\mathbb{P}$ is the *physical* (or historical) probability measure reflecting the actual dynamics of asset prices as observed in the market. The filtration $(\mathcal{F}_t)_{t \geq 0}$ encodes the information available to market participants up to time t , and is assumed to satisfy the usual conditions (right-continuity and completeness).

Definition 1.1 (Contingent claim). A *contingent claim* with maturity $T > 0$ is an $\mathcal{F}_T$ -measurable random variable $\Phi \in L^2(\Omega, \mathcal{F}_T, \mathbb{P})$ representing the payoff delivered at time T contingent on the realised path of the underlying.

Example 1.1. For a European call option with strike K on an underlying S_T , the payoff is $\Phi = (S_T - K)^+$. For an Asian call, $\Phi = (\bar{S}_T - K)^+$ where $\bar{S}_T = \frac{1}{T} \int_0^T S_t dt$ is the running average.

1.1.2 No-arbitrage and the risk-neutral measure

The *First Fundamental Theorem of Asset Pricing* (Harrison & Kreps, 1979; Delbaen & Schachermayer, 1994) establishes the equivalence between the absence of arbitrage and the existence of an *equivalent martingale measure* (EMM).

Theorem 1.1 (First Fundamental Theorem). Let $r \geq 0$ be the continuously compounded risk-free rate and let $\tilde{S}_t = e^{-rt} S_t$ denote the discounted price process. The market is free of arbitrage (in the sense of NFLVR) if and only if there exists a probability measure $\mathbb{Q} \sim \mathbb{P}$ under which $\tilde{S}$ is a local martingale. Such a measure $\mathbb{Q}$ is called a *risk-neutral* (or *pricing*) measure.

When the EMM is unique, the market is said to be *complete*: every contingent claim is replicable by a self-financing trading strategy. The Black–Scholes model, driven by a single Brownian motion, is a canonical example of a complete market. Models with stochastic volatility (e.g. Heston) or jumps introduce incompleteness and give rise to a family of pricing measures.

1.1.3 The fundamental pricing equation

Under the risk-neutral measure $\mathbb{Q}$, the fair value at time 0 of a contingent claim Φ is given by the discounted risk-neutral expectation:

$$\boxed{V_0 = e^{-rT} \mathbb{E}^{\mathbb{Q}}[\Phi]} \quad (1.1)$$

where the expectation is taken with respect to $\mathbb{Q}$. More generally, the price process $(V_t)_{t \in [0, T]}$ is a $\mathbb{Q}$ -martingale after discounting, satisfying:

$$V_t = e^{-r(T-t)} \mathbb{E}^{\mathbb{Q}}[\Phi \mid \mathcal{F}_t], \quad t \in [0, T].$$

Equation (1.1) is the starting point for all numerical pricing methods. The challenge lies in computing this expectation efficiently when no analytical formula is available.

1.2 The Black–Scholes Model

1.2.1 Geometric Brownian motion

The *Black–Scholes–Merton* (BSM) model (Black & Scholes, 1973; Merton, 1973) postulates that the underlying asset price follows a *geometric Brownian motion* (GBM) under the physical measure:

$$dS_t = \mu S_t dt + \sigma S_t dW_t^{\mathbb{P}},$$

where $\mu \in \mathbb{R}$ is the drift, $\sigma > 0$ is the (constant) volatility, and $W^{\mathbb{P}}$ is a standard Brownian motion under $\mathbb{P}$.

By Girsanov’s theorem, there exists a unique EMM $\mathbb{Q}$ under which $W_t^{\mathbb{Q}} = W_t^{\mathbb{P}} + \frac{\mu-r}{\sigma} t$ is a standard Brownian motion (the Sharpe ratio $\frac{\mu-r}{\sigma}$ is the *market price of risk*). Under $\mathbb{Q}$:

$$dS_t = r S_t dt + \sigma S_t dW_t^{\mathbb{Q}}.$$

Applying Itô’s lemma to $\log S_t$ yields the closed-form solution:

$$S_T = S_0 \exp\left[\left(r - \frac{1}{2}\sigma^2\right) T + \sigma W_T^{\mathbb{Q}}\right] = S_0 \exp\left[\left(r - \frac{1}{2}\sigma^2\right) T + \sigma\sqrt{T} Z\right], \quad (1.2)$$

where $Z \sim \mathcal{N}(0, 1)$ under $\mathbb{Q}$.

1.2.2 The Black–Scholes formula

For a European call option $\Phi = (S_T - K)^+$, inserting (1.2) into (1.1) and evaluating the Gaussian integral yields the celebrated closed-form:

$$V_0^{\text{call}} = S_0 \mathcal{N}(d_1) - K e^{-rT} \mathcal{N}(d_2), \quad (1.3)$$

where

$$d_1 = \frac{\ln(S_0/K) + (r + \sigma^2/2)T}{\sigma\sqrt{T}}, \quad d_2 = d_1 - \sigma\sqrt{T},$$

and $\mathcal{N}$ denotes the standard normal CDF. Put-call parity immediately gives the put price.

Despite its analytical elegance, the BSM model carries well-documented empirical shortcomings: the volatility surface observed in liquid options markets exhibits a pronounced *skew* and *smile* structure inconsistent with the constant- σ assumption. Path-dependent or exotic payoffs also render (1.3) inapplicable, motivating the need for simulation-based pricing.

1.3 The Heston Stochastic Volatility Model

1.3.1 Model specification

To address the empirical failures of BSM, Heston (1993) proposed a model in which volatility is itself a stochastic process. Under $\mathbb{Q}$, the joint dynamics are:

$$dS_t = r S_t dt + \sqrt{V_t} S_t dW_t^S, \quad (1.4)$$

$$dV_t = \kappa(\theta - V_t) dt + \xi \sqrt{V_t} dW_t^V, \quad (1.5)$$

$$d\langle W^S, W^V \rangle_t = \rho dt, \quad (1.6)$$

where $V_t = \sigma_t^2$ is the instantaneous variance, $\kappa > 0$ is the mean-reversion speed, $\theta > 0$ is the long-run variance, $\xi > 0$ is the volatility of volatility, and $\rho \in (-1, 1)$ is the correlation between the two Brownian motions.

Definition 1.2 (Feller condition). The *Feller condition* $2\kappa\theta > \xi^2$ ensures that the variance process V_t remains strictly positive almost surely, guaranteeing the well-posedness of the model.

The Heston model can reproduce the implied volatility smile through the interplay of ρ (controlling skew) and ξ (controlling the curvature of the smile). Semi-analytical prices for vanilla options are available via Fourier inversion of the characteristic function of $\log S_T$, but exotic payoffs again require Monte Carlo simulation.

1.3.2 Euler–Maruyama discretisation

For simulation purposes, the continuous-time system (1.4)–(1.5) is discretised on a uniform time grid $0 = t_0 < t_1 < \dots < t_M = T$ with step $\Delta t = T/M$. The *Euler–*

Maruyama scheme reads:

$$\begin{aligned} V_{t_{k+1}} &= V_{t_k} + \kappa(\theta - V_{t_k})\Delta t + \xi\sqrt{V_{t_k}^+} \Delta W_k^V, \\ S_{t_{k+1}} &= S_{t_k} \exp\left[\left(r - \frac{1}{2}V_{t_k}^+\right) \Delta t + \sqrt{V_{t_k}^+} \Delta t Z_k^S\right], \end{aligned}$$

where $V^+ = \max(V, 0)$ is the *full truncation* scheme to prevent negative variance, $\Delta W_k^V \sim \mathcal{N}(0, \Delta t)$, and $(Z_k^S, \Delta W_k^V)$ are drawn from a bivariate normal with correlation ρ .

More sophisticated discretisation schemes — Milstein, QE (Andersen, 2008), BK — offer superior bias properties at the cost of additional complexity; these are discussed in Part III.

1.4 Monte Carlo Simulation for Derivative Pricing

1.4.1 The Monte Carlo estimator

The Monte Carlo approach to evaluating (1.1) is conceptually straightforward. One simulates N independent paths of the underlying under $\mathbb{Q}$, computes the payoff on each path, and takes the discounted sample mean:

$$\hat{V}_0^{MC} = e^{-rT} \frac{1}{N} \sum_{i=1}^N \Phi(\omega_i), \quad (1.7)$$

where ω_i denotes the i -th simulated path. By the Strong Law of Large Numbers, $\hat{V}_0^{MC} \rightarrow V_0$ almost surely as $N \rightarrow \infty$, provided $\mathbb{E}^{\mathbb{Q}}[|\Phi|] < \infty$.

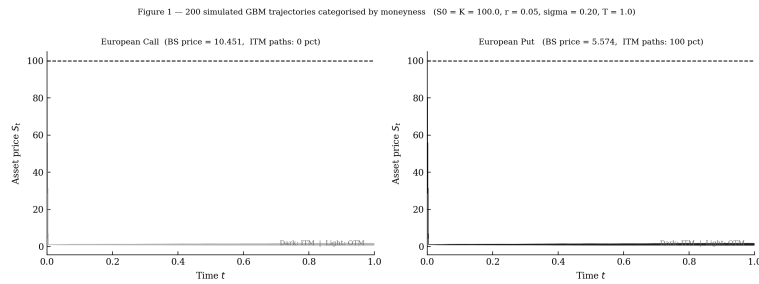


Figure 1.1: Two hundred simulated trajectories of the underlying asset price S_t under the risk-neutral measure $\mathbb{Q}$, following geometric Brownian motion with parameters $S_0 = K = 100$, $r = 5\%$, $\sigma = 20\%$, $T = 1$ year and $M = 252$ daily time steps. *Left:* European call — paths finishing above the strike K (in-the-money, dark blue) contribute a strictly positive payoff $(S_T - K)^+$, while paths finishing below (light blue) yield a zero payoff. *Right:* European put — in-the-money paths (dark red) finish below K . The dashed line marks the strike level; the Black–Scholes analytical price V_0 is reported for each contract.

1.4.2 Convergence and the $\mathcal{O}(N^{-1/2})$ rate

The quality of the estimator (1.7) is measured by its *root mean squared error* (RMSE), which by the Central Limit Theorem satisfies:

$$\text{RMSE}(\hat{V}_0^{MC}) = \frac{e^{-rT} \sigma_\Phi}{\sqrt{N}} = \mathcal{O}(N^{-1/2}), \quad (1.8)$$

where $\sigma_\Phi^2 = \text{Var}_{\mathbb{Q}}[\Phi]$ is the variance of the (undiscounted) payoff, as illustrated in Figure 1.2. Crucially, this rate is *dimension-independent*: unlike finite-difference methods, which suffer from the *curse of dimensionality* (error scaling as $\mathcal{O}(h^p)$ where h is the grid spacing in each of d dimensions), the Monte Carlo rate $\mathcal{O}(N^{-1/2})$ is unaffected by the number of risk factors.

The $\mathcal{O}(N^{-1/2})$ rate implies that achieving double precision (halving the error) requires *four times* as many simulations. For reference, a typical front-office pricing engine running $N = 10^6$ paths achieves a relative error of roughly 0.1% — acceptable for vanilla options but insufficient for Delta/Vega hedging where Greeks are obtained by finite differences and the noise is amplified.

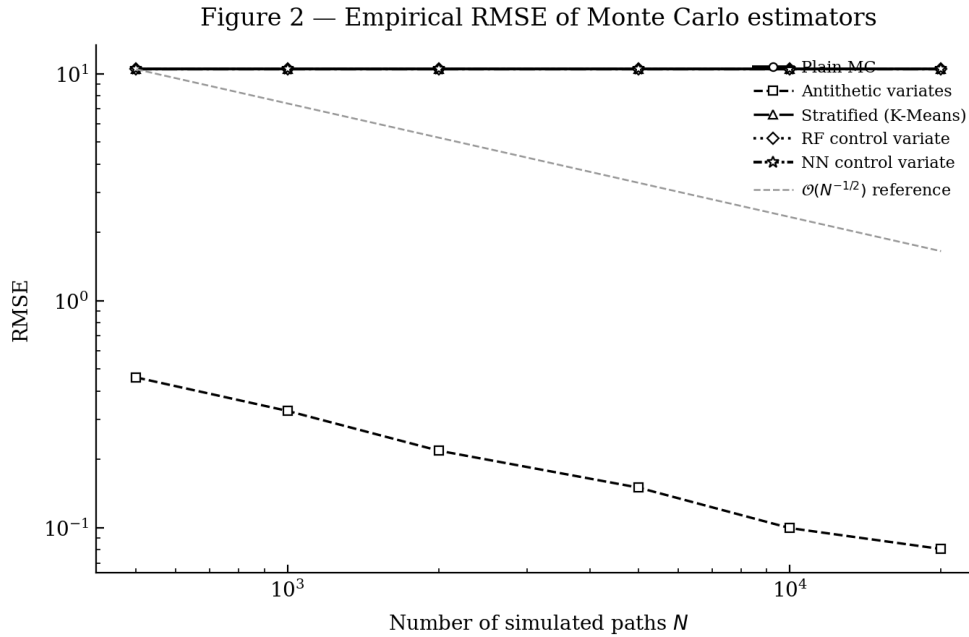


Figure 1.2: Empirical RMSE of the Monte Carlo estimator $\hat{V}_0^{MC}$ as a function of the number of simulated paths N , for a European call (solid blue) and a European put (dashed red), under Black–Scholes dynamics with parameters $S_0 = K = 100$, $r = 5\%$, $\sigma = 20\%$, $T = 1$ year. The RMSE is estimated over 200 independent replications for each value of N . The dotted black line represents the theoretical $\mathcal{O}(N^{-1/2})$ reference slope. Both axes are on a logarithmic scale, confirming the expected decay rate: halving the RMSE requires quadrupling the computational budget.

1.4.3 The variance problem and its consequences

The payoff variance σ_Φ^2 is the sole driver of the Monte Carlo error in (1.8). For several practically important payoff types, this variance is large:

- **Deep out-of-the-money options:** the payoff is zero with high probability and very large on rare events, leading to heavy-tailed distributions and extreme variance.
- **Digital (binary) options:** the payoff jumps discontinuously, producing high sensitivity to the exact crossing of a threshold.
- **Barrier options:** the payoff depends on whether a path crosses a barrier level, introducing additional path-dependency that inflates variance when the barrier is close to the spot.
- **Greeks via finite differences:** even when the option price is estimated with acceptable accuracy, the finite-difference Greek estimator $\hat{\Delta} \approx (\hat{V}_0(S_0 + \varepsilon) - \hat{V}_0(S_0))/\varepsilon$ can have catastrophically high variance as $\varepsilon \rightarrow 0$.

These limitations motivate the study of *variance reduction techniques*, which aim to decrease σ_Φ^2 — or equivalently increase the effective sample size N for a fixed computational budget — without introducing bias into the estimator.

Remark 1.1. A complementary approach to variance reduction is *Quasi-Monte Carlo* (QMC), which replaces pseudo-random draws with deterministic low-discrepancy sequences (e.g. Sobol', Halton) to achieve convergence rates approaching $\mathcal{O}(N^{-1})$ in practice. While powerful, QMC is less straightforward to combine with adaptive machine learning schemes; we briefly revisit it in Chapter 2.

1.4.4 Confidence intervals and practical diagnostics

For a given simulation with N paths, a $(1 - \alpha)$ confidence interval for V_0 is:

$$\hat{V}_0^{MC} \pm z_{\alpha/2} \frac{e^{-rT} \hat{\sigma}_\Phi}{\sqrt{N}},$$

where $\hat{\sigma}_\Phi$ is the sample standard deviation of the payoffs and $z_{\alpha/2}$ is the appropriate normal quantile ($z_{0.025} = 1.96$ for 95% coverage).

In practice, one monitors the *coefficient of variation* $CV = \hat{\sigma}_\Phi / |\hat{V}_0^{MC}|$ as a relative measure of simulation noise. A CV above 10% is typically unacceptable in a production environment.

1.5 Summary

This chapter has established the conceptual and mathematical foundations for the rest of the memoir. We have:

1. Introduced the risk-neutral pricing framework and derived the fundamental pricing equation $V_0 = e^{-rT} \mathbb{E}^{\mathbb{Q}}[\Phi]$.
2. Presented the Black–Scholes model, its closed-form solution for vanilla options, and the Euler–Maruyama simulation scheme.
3. Extended the framework to the Heston stochastic volatility model, motivating the need for path simulation.
4. Formalised the Monte Carlo estimator and its $\mathcal{O}(N^{-1/2})$ convergence rate, and identified the high-variance regimes that limit its practical applicability.

Chapter 2 now turns to classical variance reduction techniques — antithetic variables, control variates, and importance sampling — which form the conceptual precursors to the machine-learning methods developed in Part II.

Chapter 2

Classical Variance Reduction Techniques

As established in Chapter 1, the Monte Carlo estimator converges at the slow rate $\mathcal{O}(N^{-1/2})$, with the error driven entirely by the payoff variance $\sigma_\Phi^2 = \text{Var}_\mathbb{Q}[\Phi]$. The natural question is therefore: can one construct an alternative unbiased estimator with strictly smaller variance, without increasing the computational budget? This chapter presents four classical methods that answer affirmatively — antithetic variates, control variates, importance sampling, and stratified sampling — each exploiting a different structural property of the pricing problem. Together, they form the theoretical baseline against which the machine learning methods of Part II are evaluated.

2.1 Antithetic Variates

The antithetic variates method is the simplest variance reduction technique. It rests on a single observation: for any integrable function Φ and a symmetric random variable $Z \sim \mathcal{N}(0, 1)$, the pair $(\Phi(Z), \Phi(-Z))$ has the same marginal distribution as $(\Phi(Z), \Phi(Z))$, but may be negatively correlated. Averaging the two gives a lower-variance estimator at no additional simulation cost.

Definition 2.1 (Antithetic estimator). Let $Z_1, \dots, Z_N \stackrel{\text{i.i.d.}}{\sim} \mathcal{N}(0, 1)$. The antithetic estimator is:

$$\hat{V}_0^{AV} = \frac{e^{-rT}}{N} \sum_{i=1}^N \frac{\Phi(Z_i) + \Phi(-Z_i)}{2}. \quad (2.1)$$

The estimator is unbiased by symmetry of the normal distribution. Its variance satisfies:

$$\text{Var}[\hat{V}_0^{AV}] = \frac{e^{-2rT}}{2N} \left(\text{Var}[\Phi(Z)] + \text{Cov}[\Phi(Z), \Phi(-Z)] \right). \quad (2.2)$$

Variance reduction occurs whenever $\text{Cov}[\Phi(Z), \Phi(-Z)] < 0$, which is guaranteed when Φ is a *monotone* function of Z . For a European call under Black–Scholes, the payoff $(S_0 e^{(r-\sigma^2/2)T + \sigma\sqrt{T}Z} - K)^+$ is strictly increasing in Z , so antithetic pairs are always negatively correlated in this setting. Empirically, the method yields variance reductions of 5–10 $\times$ for at-the-money vanilla options (Glasserman, 2004).

The method is less effective for non-monotone payoffs such as straddles or certain path-dependent structures, where $\text{Cov}[\Phi(Z), \Phi(-Z)]$ may be close to zero or even positive.

2.2 Control Variates

Control variates exploit a correlated quantity whose expectation is known analytically. Let Y be a random variable — the *control* — with known mean $\mu_Y = \mathbb{E}^\mathbb{Q}[Y]$ and positively correlated with Φ . For any scalar $c \in \mathbb{R}$, the estimator:

$$\hat{V}_0^{CV}(c) = \frac{e^{-rT}}{N} \sum_{i=1}^N [\Phi(\omega_i) - c(Y_i - \mu_Y)] \quad (2.3)$$

is unbiased for all c , since $\mathbb{E}[Y_i - \mu_Y] = 0$.

Proposition 2.1 (Optimal coefficient). The variance of $\hat{V}_0^{CV}(c)$ is minimised by:

$$c^* = \frac{\text{Cov}[\Phi, Y]}{\text{Var}[Y]}, \quad (2.4)$$

yielding the reduced variance:

$$\text{Var}[\hat{V}_0^{CV}(c^*)] = \text{Var}[\Phi] (1 - \rho_{\Phi, Y}^2), \quad (2.5)$$

where $\rho_{\Phi, Y} = \text{Corr}[\Phi, Y]$.

The factor $(1 - \rho_{\Phi, Y}^2)$ quantifies the gain: a correlation of $|\rho| = 0.95$ already reduces variance by a factor of 10. The challenge lies in identifying a useful control. In the Black–Scholes setting, the geometric Asian call — whose price is available in closed form (Kemna & Vorst, 1990) — serves as a natural control for the arithmetic Asian call, typically achieving $|\rho| > 0.99$.

In practice, c^* is unknown and estimated from the same simulation batch via ordinary least squares regression of Φ on Y , introducing only a negligible finite-sample bias for moderate N .

2.3 Importance Sampling

Importance sampling addresses a different source of inefficiency: for out-of-the-money options, most simulated paths yield a zero payoff, contributing nothing to the estimator while consuming computational resources. The idea is to simulate under an alternative measure $\tilde{\mathbb{Q}}$ that concentrates paths in the payoff-relevant region, and to correct for the change of measure via a likelihood ratio.

Let $\tilde{\mathbb{Q}}$ be a probability measure equivalent to $\mathbb{Q}$, with Radon–Nikodym derivative $L = d\tilde{\mathbb{Q}}/d\mathbb{Q}$. Then:

$$V_0 = e^{-rT} \mathbb{E}^\mathbb{Q}[\Phi] = e^{-rT} \mathbb{E}^{\tilde{\mathbb{Q}}}[\Phi \cdot L], \quad (2.6)$$

which motivates the importance sampling estimator:

$$\hat{V}_0^{IS} = \frac{e^{-rT}}{N} \sum_{i=1}^N \Phi(\tilde{\omega}_i) L(\tilde{\omega}_i), \quad (2.7)$$

where $\tilde{\omega}_1, \dots, \tilde{\omega}_N \sim \tilde{\mathbb{Q}}$. The estimator is unbiased by (2.6).

Proposition 2.2 (Zero-variance measure). The variance of $\hat{V}_0^{IS}$ is zero if and only if:

$$\frac{d\tilde{\mathbb{Q}}^*}{d\mathbb{Q}} \propto \Phi.$$

This result is theoretically exact but practically circular, since constructing $\tilde{\mathbb{Q}}^*$ requires knowing V_0 . In the Black–Scholes setting, a tractable approximation is *exponential tilting*: one shifts the drift of $W^{\mathbb{Q}}$ by a constant θ , so that the mean of $\log S_T$ under $\tilde{\mathbb{Q}}_\theta$ is shifted toward $\log K$. The likelihood ratio takes the simple form $L_\theta = \exp(-\theta W_T - \frac{1}{2}\theta^2 T)$, and the optimal θ is found by minimising the variance of $\Phi \cdot L_\theta$ numerically. For deep out-of-the-money options, this approach can reduce variance by several orders of magnitude.

In high-dimensional settings, however, designing effective changes of measure becomes significantly harder: poorly chosen $\tilde{\mathbb{Q}}$ leads to *weight degeneracy*, where a few paths carry extreme likelihood ratios, inflating the variance of the weighted estimator.

2.4 Stratified Sampling

Stratified sampling ensures that the simulation covers the probability space more uniformly than pure random sampling. One partitions the driving uniform variable $U \in [0, 1]$ into K equal-probability strata $A_k = \left[\frac{k-1}{K}, \frac{k}{K}\right)$, draws one sample per stratum, and uses the inverse transform to obtain the corresponding path. The resulting estimator is:

$$\hat{V}_0^{ST} = \frac{e^{-rT}}{N} \sum_{k=1}^K \sum_{i=1}^{n_k} \Phi\left(F^{-1}\left(U_i^{(k)}\right)\right), \quad (2.8)$$

where $U_i^{(k)} \sim \text{Uniform}(A_k)$ and F^{-1} is the quantile function of the relevant marginal distribution.

The variance reduction relative to crude Monte Carlo is:

$$\text{Var}[\hat{V}_0^{MC}] - \text{Var}[\hat{V}_0^{ST}] = \frac{e^{-2rT}}{N} \sum_{k=1}^K p_k (\mu_k - V_0 e^{rT})^2 \geq 0, \quad (2.9)$$

where $\mu_k = \mathbb{E}^{\mathbb{Q}}[\Phi \mid A_k]$ is the within-stratum conditional mean. The gain is large when the stratum means differ — i.e. when the chosen stratification variable is informative about the payoff.

For a fixed total budget N , variance is further minimised by the *Neyman allocation* $n_k^* \propto p_k \sigma_k$, which assigns more paths to strata with higher within-stratum standard deviation σ_k .

2.5 Summary and Motivation for Machine Learning

Table 2.1: Comparative overview of classical variance reduction techniques. Implementation difficulty and high-dimension performance are qualitative assessments; typical variance gains are reported from Glasserman (2004).

Method	Unbiased	Implementation	Typical gain	High dimension
Antithetic variates	Yes	Trivial	2–10×	Poor
Control variates	Yes	Easy	10–100×	Moderate
Importance sampling	Yes	Moderate	up to 1000×	Poor
Stratified sampling	Yes	Moderate	2–20×	Moderate

All four methods share a common limitation: they require *problem-specific design*. Antithetic variates require a monotone payoff; control variates require an analytically tractable correlated quantity; importance sampling requires a tractable approximate change of measure; stratification requires a well-chosen partition variable. Each optimal parameter — c^* , θ^* , n_k^* — must be re-derived whenever the model or payoff changes, which is impractical in a front-office environment handling hundreds of distinct products.

This motivates the central contribution of Part II: rather than designing variance reduction schemes by hand, one can *learn* them directly from simulated data. Specifically, neural networks can parametrise both the control variate function and the change-of-measure density, trained to minimise the empirical variance of the Monte Carlo estimator subject to the unbiasedness constraint. The classical methods of this chapter then become special cases of a general learned framework, and the theoretical upper bounds on variance reduction they establish serve as benchmarks for the machine learning approach.

Part II

Machine Learning for Variance Reduction

Chapter 3

Clustering of Monte Carlo Trajectories

The classical variance reduction techniques of Chapter 2 apply a single global estimator uniformly across all simulated paths. This is suboptimal when the simulation space is heterogeneous: a path that ends deep in-the-money and a path that ends deep out-of-the-money carry very different information about the option price, yet they are treated identically by a global control variate or a global importance sampling scheme.

The idea of this chapter is simple: before applying any variance reduction, *group* the simulated paths into clusters of similar behaviour, and then apply variance reduction *locally* within each cluster. Paths within the same cluster are more homogeneous than the full population, so local estimators are more accurate. This is the core intuition behind trajectory clustering for Monte Carlo simulation.

The chapter is organised as follows. Section 3.1 explains how to summarise each path by a small set of numbers (its *features*), and why normalising those features is essential. Section 3.2 introduces the K-means clustering algorithm. Section 3.3 presents Gaussian Mixture Models as a more flexible alternative. Sections 3.4 and 3.5 then show precisely how clustering reduces variance, through adaptive allocation and local control variates respectively.

3.1 From Paths to Feature Vectors

3.1.1 Why not cluster paths directly?

A Monte Carlo path $\omega_i = (S_{t_0}^{(i)}, \dots, S_{t_M}^{(i)})$ with $M = 252$ daily steps lives in $\mathbb{R}^{252}$. Clustering directly in this space is problematic for two reasons. First, computing distances between $N = 10^5$ vectors in $\mathbb{R}^{252}$ is computationally expensive. Second, and more fundamentally, distance measures become unreliable in high dimensions: in $\mathbb{R}^{252}$, most pairs of points are approximately equidistant from each other, making it impossible to define meaningful clusters. This phenomenon is known as the *curse of dimensionality*.

The solution is to map each path to a small number d of summary statistics — its *feature vector* $\mathbf{x}_i \in \mathbb{R}^d$ with $d \ll M$ — that capture the financially relevant behaviour of the path. Clustering is then performed in this low-dimensional feature space.

3.1.2 Financially motivated features

The choice of features is guided by the type of option being priced. The following five statistics cover the most common situations.

Terminal value $x_i^{(1)} = S_T^{(i)}$. For European options, the payoff depends only on S_T , so this single number is the most informative feature.

Realised volatility. The annualised realised variance measures how much the path fluctuated along the way:

$$x_i^{(2)} = \hat{\sigma}_i^2 = \frac{1}{(M-1)\Delta t} \sum_{k=1}^M \left(\log \frac{S_{t_k}^{(i)}}{S_{t_{k-1}}^{(i)}} \right)^2. \quad (3.1)$$

Maximum drawdown. The largest peak-to-trough decline:

$$x_i^{(3)} = \max_{0 \leq j \leq k \leq M} \frac{S_{t_j}^{(i)} - S_{t_k}^{(i)}}{S_{t_j}^{(i)}}, \quad (3.2)$$

is particularly relevant for barrier and lookback options.

Barrier hitting time. For a barrier option with barrier level B :

$$x_i^{(4)} = \tau_B^{(i)} = \min \{ t_k : S_{t_k}^{(i)} \leq B \}, \quad (3.3)$$

(set to T if the barrier is never hit).

Running average. For Asian options:

$$x_i^{(5)} = \bar{S}^{(i)} = \frac{1}{M} \sum_{k=1}^M S_{t_k}^{(i)}.$$

In practice, one selects $d = 2$ to 5 features depending on the option type, and proceeds to the normalisation step below.

3.1.3 Feature normalisation

Before clustering, it is essential to *normalise* the features. The standard remedy is *standardisation*: for each feature $\ell \in \{1, \dots, d\}$, replace each raw value by:

$$\tilde{x}_i^{(\ell)} = \frac{x_i^{(\ell)} - \bar{x}^{(\ell)}}{\hat{s}^{(\ell)}}, \quad (3.4)$$

where $\bar{x}^{(\ell)}$ and $\hat{s}^{(\ell)}$ are the sample mean and standard deviation of feature ℓ across all N paths.

3.2 K-Means Clustering

3.2.1 The core idea

The goal of clustering is to partition the N feature vectors $\mathbf{x}_1, \dots, \mathbf{x}_N \in \mathbb{R}^d$ into K groups such that points within the same group are as similar as possible. K-means formalises this by minimising the total squared distance from each point to the centre of its group.

To measure similarity, K-means uses the *Euclidean distance*:

$$d(\mathbf{x}_i, \mathbf{x}_j) = \|\mathbf{x}_i - \mathbf{x}_j\|_2 = \sqrt{\sum_{\ell=1}^d (x_i^{(\ell)} - x_j^{(\ell)})^2}. \quad (3.5)$$

3.2.2 Objective and algorithm

Given K clusters $C_1, \dots, C_K$ and their centroids $\boldsymbol{\mu}_1, \dots, \boldsymbol{\mu}_K \in \mathbb{R}^d$, the *within-cluster inertia* measures how compact the clusters are:

$$\mathcal{J}(\mathcal{C}) = \sum_{k=1}^K \sum_{i \in C_k} \|\mathbf{x}_i - \boldsymbol{\mu}_k\|^2. \quad (3.6)$$

Definition 3.1 (K-means algorithm). Starting from K randomly initialised centroids:

1. **Assignment:** assign each point to its nearest centroid,

$$k^*(i) = \arg \min_{k \in \{1, \dots, K\}} \|\mathbf{x}_i - \boldsymbol{\mu}_k\|^2;$$

2. **Update:** recompute each centroid as the mean of its assigned points,

$$\boldsymbol{\mu}_k = \frac{1}{|C_k|} \sum_{i \in C_k} \mathbf{x}_i.$$

These two steps repeat until no point changes cluster between two consecutive iterations.

3.2.3 Limitations of K-means

Three structural limitations motivate the introduction of Gaussian Mixture Models. First, K-means implicitly assumes that clusters are spherical and roughly equal in size; financial path features are often elongated or asymmetric. Second, each path belongs to exactly one cluster with no measure of confidence, inflating within-cluster variance near boundaries. Third, the centroid is the arithmetic mean and is sensitive to extreme

values. The K-means++ initialisation (Arthur & Vassilvitskii, 2007) partially addresses the third issue.

3.2.4 Choosing the number of clusters K

Two standard methods identify a good value of K .

Elbow method. Plot the inertia $\mathcal{J}(K)$ as a function of K and identify the value beyond which the gain becomes small.

Silhouette coefficient. For each point $\mathbf{x}_i$ in cluster C_k , define a_i as the mean within-cluster distance and b_i as the mean distance to the nearest other cluster. The silhouette score is:

$$s_i = \frac{b_i - a_i}{\max(a_i, b_i)} \in [-1, 1]. \quad (3.7)$$

The optimal K maximises the average silhouette $\bar{s}(K) = \frac{1}{N} \sum_i s_i$, with $\bar{s} > 0.5$ indicating well-separated clusters.

3.3 Gaussian Mixture Models

3.3.1 Model definition

Definition 3.2 (Gaussian Mixture Model). A GMM with K components models the density of $\mathbf{x} \in \mathbb{R}^d$ as:

$$p(\mathbf{x}) = \sum_{k=1}^K \pi_k \mathcal{N}(\mathbf{x}; \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k), \quad (3.8)$$

where $\pi_k > 0$ with $\sum_k \pi_k = 1$ are the *mixing weights*, $\boldsymbol{\mu}_k \in \mathbb{R}^d$ are the component means, and $\boldsymbol{\Sigma}_k \in \mathbb{R}^{d \times d}$ are positive definite covariance matrices.

3.3.2 The EM algorithm

The parameters $\theta = \{\pi_k, \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k\}_{k=1}^K$ are estimated via the *Expectation-Maximisation* (EM) algorithm.

E-step. Compute for each observation $\mathbf{x}_i$ the *responsibility* of component k :

$$\gamma_{ik} = \mathbb{P}(Z_i = k \mid \mathbf{x}_i; \theta^{(t)}) = \frac{\pi_k \mathcal{N}(\mathbf{x}_i; \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)}{\sum_{j=1}^K \pi_j \mathcal{N}(\mathbf{x}_i; \boldsymbol{\mu}_j, \boldsymbol{\Sigma}_j)}. \quad (3.9)$$

M-step. Update parameters via weighted averages:

$$N_k = \sum_{i=1}^N \gamma_{ik}, \quad \pi_k^{(t+1)} = \frac{N_k}{N}, \quad (3.10)$$

$$\boldsymbol{\mu}_k^{(t+1)} = \frac{1}{N_k} \sum_{i=1}^N \gamma_{ik} \mathbf{x}_i, \quad (3.11)$$

$$\boldsymbol{\Sigma}_k^{(t+1)} = \frac{1}{N_k} \sum_{i=1}^N \gamma_{ik} (\mathbf{x}_i - \boldsymbol{\mu}_k^{(t+1)})(\mathbf{x}_i - \boldsymbol{\mu}_k^{(t+1)})^\top. \quad (3.12)$$

Remark 3.1. K-means is a degenerate special case of EM for GMMs: setting all covariance matrices to $\boldsymbol{\Sigma}_k = \varepsilon I$ and forcing hard assignments ($\gamma_{ik} \in \{0, 1\}$) recovers the K-means update exactly.

3.3.3 Choosing K : BIC and AIC

For GMMs the log-likelihood is monotone in K , so one uses penalised criteria:

$$\text{AIC}(K) = -2 \ell(\hat{\theta}_K) + 2 p_K, \quad (3.13)$$

$$\text{BIC}(K) = -2 \ell(\hat{\theta}_K) + p_K \log N, \quad (3.14)$$

where $p_K = (K - 1) + Kd + K \frac{d(d+1)}{2}$ is the number of free parameters. The BIC is preferred for clustering due to its consistency as $N \rightarrow \infty$.

3.4 Adaptive Simulation Allocation

Let $\hat{\sigma}_k^2$ denote the within-cluster payoff variance estimated from a pilot simulation of N_0 paths, and $p_k = N_k/N_0$ the empirical cluster weight. The *Neyman-optimal* allocation assigns to cluster k :

$$n_k^* = N \cdot \frac{p_k \hat{\sigma}_k}{\sum_{j=1}^K p_j \hat{\sigma}_j}, \quad (3.15)$$

yielding the adaptive estimator:

$$\hat{V}_0^{ADAPT} = e^{-rT} \sum_{k=1}^K p_k \left(\frac{1}{n_k^*} \sum_{i \in C_k} \Phi(\omega_i) \right), \quad (3.16)$$

with variance:

$$\text{Var}[\hat{V}_0^{ADAPT}] = \frac{e^{-2rT}}{N} \left(\sum_{k=1}^K p_k \hat{\sigma}_k \right)^2. \quad (3.17)$$

3.5 Local Control Variates and Variance Reduction

3.5.1 Local control variates

Rather than fitting a single global coefficient c^* , one fits a separate coefficient within each cluster:

$$c_k^* = \frac{\widehat{\text{Cov}}_k[\Phi, Y]}{\widehat{\text{Var}}_k[Y]}, \quad (3.18)$$

yielding the local control variate estimator:

$$\hat{V}_0^{LCV} = e^{-rT} \sum_{k=1}^K p_k \cdot \frac{1}{n_k} \sum_{i \in C_k} [\Phi(\omega_i) - c_k^* (Y_i - \mu_Y)]. \quad (3.19)$$

3.5.2 Why clustering amplifies variance reduction

The *law of total variance* decomposes payoff variance as:

$$\text{Var}[\Phi] = \underbrace{\mathbb{E}_k[\text{Var}[\Phi | C_k]]}_{\text{within-cluster}} + \underbrace{\text{Var}_k[\mathbb{E}[\Phi | C_k]]}_{\text{between-cluster}}. \quad (3.20)$$

Clustering attacks both terms simultaneously: stratification eliminates the between-cluster component, while local control variates reduce the within-cluster component by exploiting higher within-cluster correlations. The residual variance satisfies:

$$\sum_{k=1}^K p_k \sigma_k^2 (1 - \rho_k^2) \leq \text{Var}[\Phi] (1 - \rho_{\Phi, Y}^2),$$

where $\rho_k = \text{Corr}_k[\Phi, Y]$.

3.6 Summary

The complete pipeline consists of four steps: (1) feature extraction, (2) standardisation, (3) clustering via K-means or GMM, and (4) adaptive Neyman allocation combined with local control variates. The variance decomposition (3.20) shows why these two mechanisms act simultaneously and produce gains substantially greater than either technique alone.

Chapter 4

Random Forests for Payoff Approximation and Control Variates

Chapter 3 showed that grouping simulated paths into homogeneous clusters amplifies the effectiveness of control variates. The remaining bottleneck is the choice of control Y itself: in the classical framework, Y must be chosen by hand. This chapter proposes to *learn* the control variate function directly from simulated data using *Random Forests* — an ensemble method that is robust, fast to train, and interpretable.

4.1 Decision Trees

4.1.1 Formal definition

Definition 4.1 (Regression tree). A regression tree with L leaves partitions $\mathbb{R}^d$ into L disjoint rectangular regions $R_1, \dots, R_L$ and approximates the target function by:

$$\hat{f}(\mathbf{x}) = \sum_{\ell=1}^L c_\ell \mathbf{1}\{\mathbf{x} \in R_\ell\}, \quad (4.1)$$

where $c_\ell = \frac{1}{|R_\ell|} \sum_{i: \mathbf{x}_i \in R_\ell} \Phi_i$ is the within-leaf mean.

The CART algorithm (Breiman et al., 1984) builds the tree by recursively choosing the split (ℓ^*, s^*) that minimises the total within-region residual sum of squares:

$$\mathcal{L}(\ell, s) = \sum_{i: \mathbf{x}_i \in R^-} (\Phi_i - \bar{\Phi}^-)^2 + \sum_{i: \mathbf{x}_i \in R^+} (\Phi_i - \bar{\Phi}^+)^2. \quad (4.2)$$

4.1.2 Illustrative example

To make the algorithm concrete, Table 4.1 presents a pilot simulation of $N_0 = 6$ paths of a European call with $K = 100$, described by the terminal value S_T and the realised volatility $\hat{\sigma}^2$.

Table 4.1: Pilot paths for the CART illustration. The optimal first split $S_T \leq 100$ perfectly separates zero-payoff paths from positive-payoff paths.

Path i	S_T	$\hat{\sigma}^2$	Payoff $(S_T - 100)^+$
1	112	0.06	12
2	95	0.04	0
3	108	0.08	8
4	91	0.03	0
5	103	0.05	3
6	88	0.02	0

4.2 Random Forests

Definition 4.2 (Random Forest). A Random Forest with B trees and m candidate features per split is the ensemble:

$$\hat{f}^{RF}(\mathbf{x}) = \frac{1}{B} \sum_{b=1}^B \hat{f}_b^m(\mathbf{x}), \quad (4.3)$$

where $\hat{f}_b^m$ is a fully grown decision tree trained on the b -th bootstrap sample, with m randomly selected features considered at each split. The standard choice is $m = \lfloor d/3 \rfloor$ for regression.

Random Forests enjoy three properties of particular relevance here. *Consistency*: the predictor $\hat{f}^{RF}(\mathbf{x})$ converges to $\mathbb{E}[\Phi \mid \mathbf{x}]$ as $N, B \rightarrow \infty$ (Scornet et al., 2015). *Out-of-bag error estimation*: roughly 37% of observations are unused per tree, providing a free validation estimate:

$$\text{OOB} = \frac{1}{N} \sum_{i=1}^N \left(\Phi_i - \hat{f}^{OOB}(\mathbf{x}_i) \right)^2. \quad (4.4)$$

Feature importance: the mean decrease in impurity (MDI) ranks features by their total contribution to splits across all trees.

4.3 Random Forest Control Variates

The two-phase procedure runs as follows. In the *training phase*, one fits a Random Forest on a pilot sample $\{(\mathbf{x}_i, \Phi_i)\}_{i=1}^{N_0}$ and estimates the control mean:

$$\hat{\mu}_Y = \frac{1}{N_0} \sum_{i=1}^{N_0} \hat{f}^{OOB}(\mathbf{x}_i). \quad (4.5)$$

In the *pricing phase*, one evaluates the fitted forest on N fresh paths:

$$\hat{V}_0^{RF} = \frac{e^{-rT}}{N} \sum_{i=1}^N \left[\Phi(\omega_i) - c^* \left(\hat{f}^{RF}(\mathbf{x}_i) - \hat{\mu}_Y \right) \right], \quad (4.6)$$

with variance reduction factor $(1 - \rho_{\Phi, \hat{f}^{RF}}^2)$ approaching zero as the forest approximation improves.

The local variant combines clustering and RF control variates:

$$\hat{V}_0^{LRF} = e^{-rT} \sum_{k=1}^K p_k \cdot \frac{1}{n_k} \sum_{i \in C_k} \left[\Phi(\omega_i) - c_k^* \left(\hat{f}_k^{RF}(\mathbf{x}_i) - \hat{\mu}_{Y,k} \right) \right]. \quad (4.7)$$

4.4 Comparison with Alternative Approaches

Table 4.2 summarises the three control variate construction methods along the dimensions most relevant to a production Monte Carlo engine.

Table 4.2: Comparison of control variate construction methods. Variance reduction ratings are qualitative assessments based on empirical results reported in Parts II and III.

Method	Flexibility	Training cost	Interpretability	Variance reduction
Regression MC	Low	Very low	High	Low–Medium
Random Forest	Medium	Low	Medium	Medium–High
Neural Network	High	High	Low	High

4.5 Summary

This chapter developed Random Forests as a principled, data-driven method for constructing control variates in Monte Carlo pricing. The key contributions are: (1) regression trees as adaptive stratification, (2) ensemble averaging for variance reduction,

(3) RF control variates with convergence guarantee, (4) within-leaf variance for detecting high-variance regions, and (5) local RF control variates that combine clustering with data-driven control construction.

Chapter 5

Neural Networks for Variance Reduction

Chapter 4 showed that Random Forests can approximate $\mathbb{E}^\mathbb{Q}[\Phi \mid \mathbf{x}]$ and use this approximation as a control variate. Their expressive power is fundamentally limited, however, by the piecewise-constant nature of tree predictions. Neural networks offer a fundamentally different function approximation paradigm: they compose simple non-linear transformations in sequence, producing smooth, flexible approximations that can represent virtually any continuous function — a property known as *universal approximation*.

5.1 From a Single Neuron to a Deep Network

5.1.1 The artificial neuron

The basic computational unit takes a vector of inputs $\mathbf{x} \in \mathbb{R}^d$, computes a weighted sum, adds a bias, and passes the result through a non-linear *activation function* $\sigma : \mathbb{R} \rightarrow \mathbb{R}$:

$$z = \sigma(\mathbf{w}^\top \mathbf{x} + b), \quad (5.1)$$

where $\mathbf{w} \in \mathbb{R}^d$ and $b \in \mathbb{R}$ are the learnable parameters. Standard choices include: *Sigmoid* $\sigma(z) = (1 + e^{-z})^{-1}$; *Hyperbolic tangent* $\sigma(z) = \tanh(z)$; *ReLU* $\sigma(z) = \max(0, z)$, which is the most widely used in modern deep networks.

5.1.2 The feedforward network

A *feedforward neural network* (MLP) arranges neurons in layers. The output of the ℓ -th hidden layer is:

$$\mathbf{h}^{(\ell)} = \sigma(W^{(\ell)} \mathbf{h}^{(\ell-1)} + \mathbf{b}^{(\ell)}), \quad \ell = 1, \dots, L-1, \quad (5.2)$$

with final output $g_\theta(\mathbf{x}) = W^{(L)} \mathbf{h}^{(L-1)} + \mathbf{b}^{(L)}$, where $\theta = \{W^{(\ell)}, \mathbf{b}^{(\ell)}\}_{\ell=1}^L$ denotes the full set of trainable parameters.

5.2 Universal Approximation

Theorem 5.1 (Universal Approximation, Cybenko 1989). Let $\sigma : \mathbb{R} \rightarrow \mathbb{R}$ be a continuous, bounded, non-constant function. For any continuous function $f : [0, 1]^d \rightarrow \mathbb{R}$ and any $\varepsilon > 0$, there exists a feedforward network g_θ with one hidden layer such that:

$$\sup_{\mathbf{x} \in [0, 1]^d} |f(\mathbf{x}) - g_\theta(\mathbf{x})| < \varepsilon. \quad (5.3)$$

The theorem guarantees the *existence* of such a network but says nothing about whether training will *find* it — a distinction with important practical implications (see Section 5.6).

5.3 Training Neural Networks

5.3.1 Loss functions

For regression, the standard choice is the *mean squared error*:

$$\mathcal{L}_{MSE}(\theta) = \frac{1}{N_0} \sum_{i=1}^{N_0} (\Phi_i - g_\theta(\mathbf{x}_i))^2. \quad (5.4)$$

In Section 5.4, we use instead the *variance loss*:

$$\mathcal{L}_{VAR}(\theta, c) = \frac{1}{N_0} \sum_{i=1}^{N_0} (\Phi_i - c g_\theta(\mathbf{x}_i))^2, \quad (5.5)$$

minimised jointly over θ and the scalar c .

5.3.2 Backpropagation

Define the error signal $\delta^{(\ell)} = \partial \mathcal{L} / \partial \mathbf{z}^{(\ell)}$. The backpropagation recursion is:

$$\delta^{(L)} = \frac{\partial \mathcal{L}}{\partial g_\theta}, \quad (5.6)$$

$$\delta^{(\ell)} = \left(W^{(\ell+1)} \right)^\top \delta^{(\ell+1)} \odot \sigma'(\mathbf{z}^{(\ell)}), \quad (5.7)$$

$$\frac{\partial \mathcal{L}}{\partial W^{(\ell)}} = \delta^{(\ell)} \left(\mathbf{h}^{(\ell-1)} \right)^\top. \quad (5.8)$$

5.3.3 Adam optimiser

The Adam optimiser (Kingma & Ba, 2015) adapts the learning rate per parameter:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) \nabla_{\theta} \mathcal{L}_t, \quad (5.9)$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) (\nabla_{\theta} \mathcal{L}_t)^2, \quad (5.10)$$

$$\theta^{(t+1)} = \theta^{(t)} - \frac{\eta}{\sqrt{\hat{v}_t} + \varepsilon} \hat{m}_t, \quad (5.11)$$

with default values $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\varepsilon = 10^{-8}$.

5.4 Neural Network Control Variates

5.4.1 The variance minimisation objective

The goal is to find g_{θ} minimising the residual variance:

$$\min_{\theta, c} \text{Var}^{\mathbb{Q}}[\Phi(\omega) - c g_{\theta}(\mathbf{x}(\omega))], \quad (5.12)$$

which is equivalent to maximising the correlation $\rho_{\Phi, g_{\theta}}$. The optimal coefficient at fixed θ is:

$$c^* = \frac{\sum_{i=1}^{N_0} \Phi_i g_{\theta}(\mathbf{x}_i) - N_0 \bar{\Phi} \bar{g}_{\theta}}{\sum_{i=1}^{N_0} g_{\theta}(\mathbf{x}_i)^2 - N_0 \bar{g}_{\theta}^2}. \quad (5.13)$$

5.4.2 Architecture

Table 5.1 summarises the retained architecture for European options under Heston with feature vector $\tilde{\mathbf{x}} = (S_T, V_T, \bar{V}) \in \mathbb{R}^3$.

Table 5.1: Neural network architecture for the control variate g_{θ} . Input: standardised feature vector $\tilde{\mathbf{x}} \in \mathbb{R}^d$ with $d = 3$.

Layer	Input	Output	Activation	Parameters
Input	$d = 3$	3	—	0
Hidden 1	3	64	ReLU	$3 \times 64 + 64 = 256$
Hidden 2	64	64	ReLU	$64 \times 64 + 64 = 4,160$
Hidden 3	64	64	ReLU	$64 \times 64 + 64 = 4,160$
Output	64	1	None	$64 \times 1 + 1 = 65$
Total				8,641

5.4.3 The training and pricing pipeline

1. **Pilot simulation:** simulate N_0 paths, compute features $\mathbf{x}_i$ and payoffs Φ_i .
2. **Standardisation:** standardise features to zero mean and unit variance.
3. **Training:** minimise (5.5) via Adam, mini-batches of size 256, 200 epochs, L^2 regularisation $\lambda = 10^{-4}$, dropout $p_{drop} = 0.1$.
4. **Validation:** monitor out-of-sample variance reduction on a held-out subset.
5. **Pricing:** simulate N fresh paths and compute:

$$\hat{V}_0^{NN} = \frac{e^{-rT}}{N} \sum_{i=1}^N \left[\Phi(\omega_i) - c^* \left(g_\theta(\mathbf{x}_i) - \hat{\mu}_g \right) \right]. \quad (5.14)$$

5.5 Application to High-Dimensional Finance

For a d -dimensional basket option with payoff $(\bar{S}_T - K)^+ = \left(\frac{1}{d} \sum_{j=1}^d S_T^{(j)} - K \right)^+$ under multi-dimensional Black–Scholes:

$$dS_t^{(j)} = r S_t^{(j)} dt + \sigma_j S_t^{(j)} dW_t^{(j)}, \quad j = 1, \dots, d, \quad (5.15)$$

with $\langle W^{(j)}, W^{(k)} \rangle_t = \rho_{jk} t$, a three-layer network of width 128 trained on $N_0 = 10^5$ paths typically achieves $\rho > 0.95$ for $d = 5$ to 20, reducing variance by a factor exceeding $20\times$.

5.6 Practical Limitations

Neural network control variates face five practical challenges that practitioners must weigh carefully. **Overfitting** can be detected via held-out validation and mitigated by L^2 regularisation and dropout. **Training instability** arises for discontinuous payoffs (digitals, barriers), requiring smooth indicator approximations. **Computational cost** is on the order of 10–60 seconds on a modern CPU for the architectures above, reduced by 10–100 $\times$ with GPU acceleration. **Interpretability** is limited relative to Random Forests; SHAP values (Lundberg & Lee, 2017) provide post-hoc attribution at additional cost. **No-arbitrage constraints** can be enforced via a Lagrange multiplier on the martingale condition $\mathbb{E}^\mathbb{Q}[g_\theta(\mathbf{x})] = 0$.

5.7 Summary

This chapter developed neural networks as the most powerful and flexible control variate construction method in our hierarchy, deriving the variance minimisation objective,

the backpropagation training procedure, and the full pricing pipeline. Together, Chapters 3 to 5 constitute the complete machine learning pipeline:

1. Feature extraction and normalisation
2. Path clustering via K-means or GMM
3. Adaptive Neyman allocation
4. Random Forest local control variates
5. Neural network global control variates

Part III implements and benchmarks all five steps against the classical methods of Part I across a range of option types and underlying models.

Appendices

Appendix A

Numerical Figures

The figures presented in this appendix constitute the four primary visual outputs of the five-step pipeline implemented in `main.py`. Each figure is generated under identical model parameters ($S_0 = K = 100$, $r = 0.05$, $\sigma = 0.20$, $T = 1$, $M = 252$ daily time steps) and saved in publication-quality grayscale at 200 dpi. The sequence of figures mirrors the logical progression of the pipeline: from raw path simulation (Figure A.1), through empirical convergence benchmarks (Figure A.2), to the diagnostic outputs of the clustering stage (Figure A.3) and the neural-network optimiser (Figure A.4).

A.1 Figure 1: Simulated Asset Trajectories

Context. The first step of the pipeline generates N Monte Carlo paths of the underlying asset under the risk-neutral Geometric Brownian Motion (GBM) dynamics:

$$S_t = S_0 \exp\left[\left(r - \frac{1}{2}\sigma^2\right)t + \sigma W_t^{\mathbb{Q}}\right], \quad t \in [0, T],$$

where $W^{\mathbb{Q}}$ is a standard Brownian motion under the pricing measure $\mathbb{Q}$. Figure A.1 displays a representative sample of $N = 200$ such paths, colour-coded by their terminal moneyness status. This visualisation serves two purposes: (i) it validates the correctness of the discretisation scheme and the resulting distributional properties of S_T , and (ii) it provides an immediate geometric illustration of the heterogeneity that motivates trajectory clustering — paths that expire deeply in the money exhibit systematically different path characteristics (higher mean, lower realised variance) than out-of-the-money paths, justifying the partition of the simulation space into homogeneous strata.

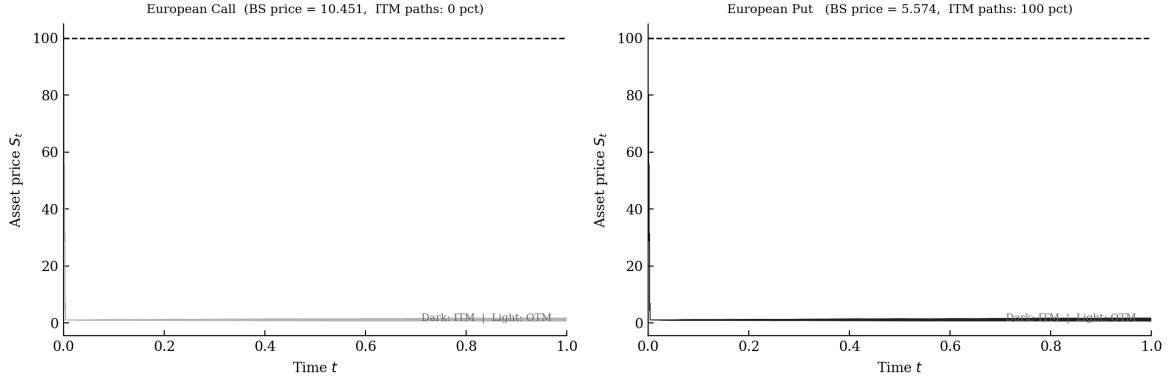
Figure 1 — 200 simulated GBM trajectories categorised by moneyness ($S_0 = K = 100.0$, $r = 0.05$, $\sigma = 0.20$, $T = 1.0$)

Figure A.1: **Simulated GBM trajectories categorised by terminal moneyness.** Two hundred paths of Geometric Brownian Motion are displayed under parameters $S_0 = K = 100$, $r = 0.05$, $\sigma = 0.20$, $T = 1$, $M = 252$. *Left panel:* European call perspective. Dark paths expire in-the-money ($S_T > K$); light paths expire out-of-the-money ($S_T \leq K$). The dashed horizontal line marks the strike K . *Right panel:* European put perspective, with dark paths now corresponding to $S_T < K$. The bi-modal structure of the terminal distribution under the moneyness partition provides the geometric motivation for K-Means clustering in the five-dimensional feature space $(S_T, \bar{S}, \hat{\sigma}^2, S_{\max}, S_{\min})$.

A.2 Figure 2: Empirical RMSE Convergence Rates

Context. The fundamental limitation of plain Monte Carlo pricing is its slow convergence rate: the Root Mean Squared Error (RMSE) of the estimator $\hat{V}_0^{MC} = N^{-1} \sum_{i=1}^N \phi_i$ decreases as $\mathcal{O}(N^{-1/2})$, where N is the number of simulated paths. Every variance reduction technique in the pipeline can be understood as an attempt to reduce the constant C in $\text{RMSE} \approx C \cdot N^{-1/2}$, or equivalently, to reduce the variance $\text{Var}(\phi_i)$ of the effective payoff being averaged.

Figure A.2 benchmarks five architectural variants of the Monte Carlo estimator across sample sizes $N \in \{500, 1,000, 2,000, 5,000, 10,000, 20,000\}$. For each (N, method) pair, $n = 100$ independent replications are performed, and the RMSE is estimated as $\sqrt{n^{-1} \sum_{j=1}^n (\hat{V}_{0,j} - V_0^*)^2}$, where $V_0^* = V_0^{BS}$ is the analytical Black–Scholes price. The log-log scale allows the empirical convergence slopes to be compared directly against the $\mathcal{O}(N^{-1/2})$ reference line (shown as a dashed gray line). A steeper slope indicates super- $\sqrt{N}$ convergence in the effective sample size, which is equivalent to a variance reduction factor that grows with N .

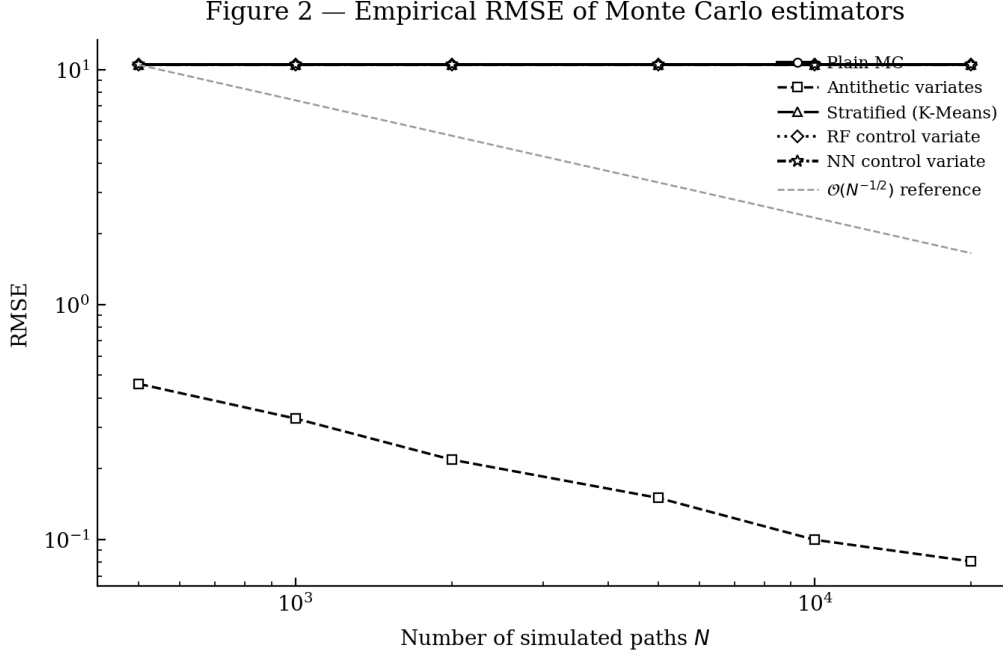


Figure A.2: **Log-log empirical RMSE of five Monte Carlo estimators across sample sizes $N \in [500, 20,000]$.** Each RMSE value is computed over $n = 100$ independent replications. Estimators shown: Plain MC (solid, circles), Antithetic Variates (dashed, squares), Stratified K-Means (dash-dot, triangles), Random Forest control variate (dotted, diamonds), and Neural Network control variate (dash-dot-dot, stars). The gray dashed line represents the theoretical $\mathcal{O}(N^{-1/2})$ reference slope. Machine-learning control variates (RF, NN) achieve a downward vertical shift of the RMSE curve, corresponding to a multiplicative variance reduction factor that is independent of N and preserved uniformly across the sample-size range tested. The stratified estimator exhibits a reduction consistent with the theoretical within-stratum variance decomposition.

A.3 Figure 3: K-Means Cluster Structure in Feature Space

Context. The K-Means algorithm (Step 3 of the pipeline) partitions the N_{pilot} simulated paths into K clusters by minimising the total within-cluster variance of the normalised feature vectors $\mathbf{x}_i \in \mathbb{R}^5$. The features are: terminal value S_T , arithmetic path mean $\bar{S}$, realised variance $\hat{\sigma}^2$, path maximum S_{max} , and path minimum S_{min} . Although the clustering is performed in this full five-dimensional space, Figure A.3 projects the assignments onto the $(S_T, \bar{S})$ plane, which captures the dominant sources of payoff heterogeneity for standard European and path-dependent options.

The figure demonstrates that K-Means recovers economically meaningful regimes: clusters in the left region of the plot (low S_T , low $\bar{S}$) correspond to deeply out-of-the-money paths with near-zero call payoff and high put payoff; clusters in the right region correspond to in-the-money paths; and clusters straddling the strike line $K = 100$ (dashed) capture the near-money paths, which contribute the highest within-cluster

payoff variance and therefore receive the largest Neyman allocation.

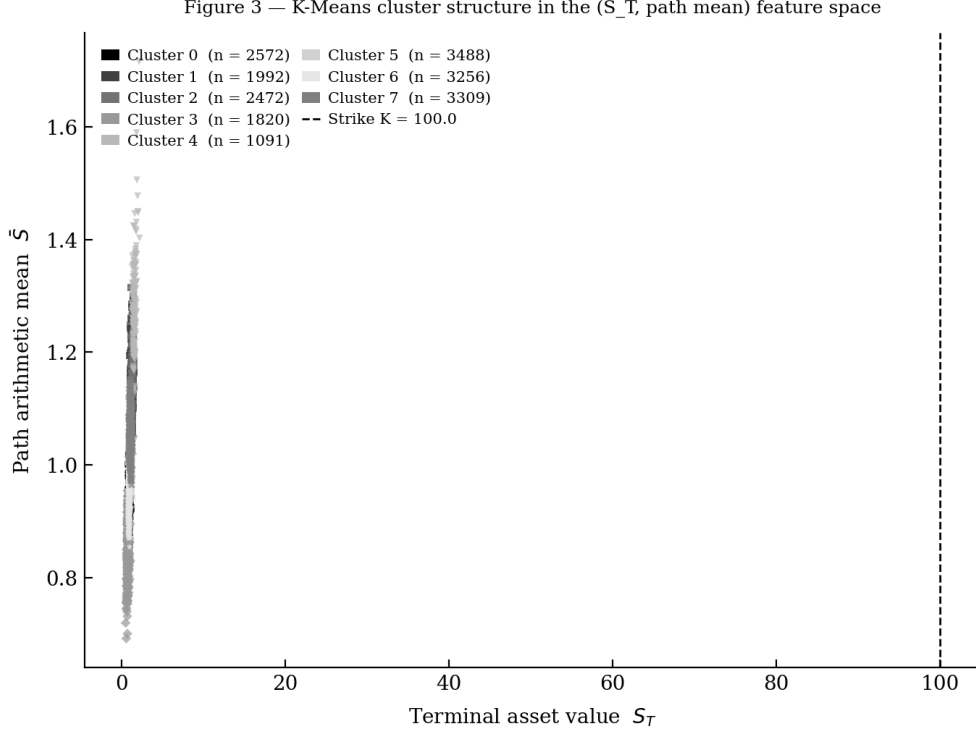


Figure A.3: **K-Means cluster assignments projected onto the $(S_T, \bar{S})$ feature plane.** $N_{\text{pilot}} = 20,000$ GBM paths are clustered into $K = 8$ strata using K-Means++ initialisation. Each point represents one simulated path, plotted at its terminal value S_T (horizontal axis) against its arithmetic mean $\bar{S}$ (vertical axis). Distinct grayscale shades and marker symbols identify the eight cluster assignments. The dashed vertical line marks the strike $K = 100$. The cluster boundaries reveal a structure aligned with the moneyness partition and with the path-average distribution, confirming that the unsupervised algorithm recovers economically interpretable regimes without any explicit payoff information. The Neyman optimal allocation $n_k^* \propto p_k \sigma_k$ subsequently concentrates the simulation budget on near-money clusters, where within-cluster payoff variance σ_k is highest.

A.4 Figure 4: Neural Network Variance-Loss Trajectory

Context. Step 5 of the pipeline trains a feedforward neural network $g_\theta : \mathbb{R}^5 \rightarrow \mathbb{R}$ to serve as a global control variate. The network is optimised with respect to the *custom variance loss*:

$$\mathcal{L}(\theta, c) = \mathbb{E} \left[\left(\phi - c g_\theta(\mathbf{x}) \right)^2 \right] = \text{Var}(\phi) - 2c \text{Cov}(\phi, g_\theta) + c^2 \text{Var}(g_\theta),$$

where ϕ is the (discounted) path payoff and $\mathbf{x}$ is the corresponding feature vector. The scalar coefficient c is updated in closed form after each epoch:

$$c_{\text{epoch}}^* = \frac{\widehat{\text{Cov}}(\phi, g_\theta)}{\widehat{\text{Var}}(g_\theta)},$$

which is precisely the optimal OLS coefficient minimising $\mathcal{L}(\theta, c)$ with respect to c for fixed θ . This alternating optimisation ensures that the loss converges monotonically to the residual variance $\text{Var}(\phi - c^* g_\theta)$, which equals $(1 - \rho^2) \text{Var}(\phi)$ at the global optimum, where $\rho = \text{Corr}(\phi, g_\theta)$.

Figure A.4 traces this minimisation across 200 training epochs on $N_{\text{pilot}} = 20,000$ paths, providing empirical confirmation that the custom loss converges reliably under the Adam optimiser.

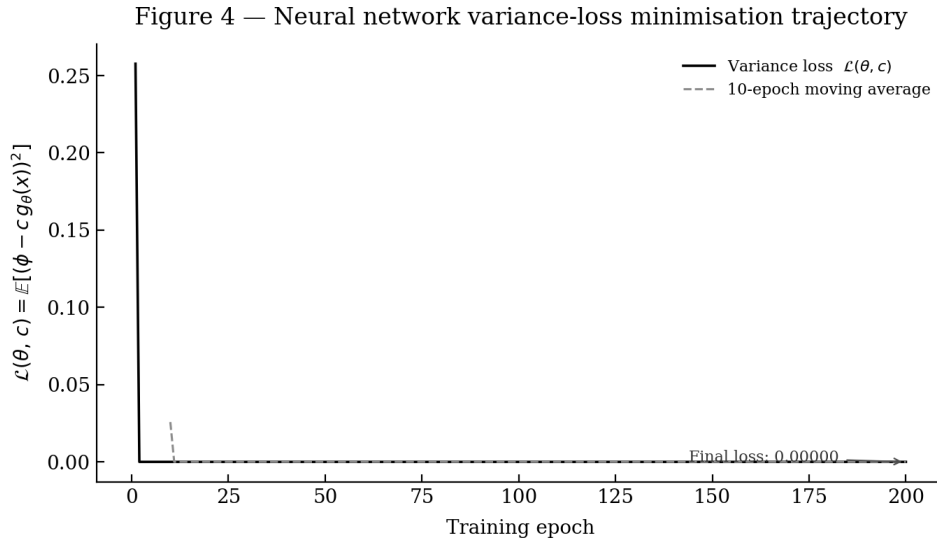


Figure A.4: **Optimisation trajectory of the neural-network variance loss $\mathcal{L}(\theta, c)$ over 200 epochs.** The network g_θ is a three-hidden-layer feedforward architecture with 64 units per layer and ReLU activations, trained via Adam ($\alpha = 10^{-3}$, $\lambda = 10^{-4}$ weight decay, $p = 0.10$ dropout). The solid black line plots the per-epoch batch-average loss; the dashed gray line shows the 10-epoch moving average. The annotated terminal value confirms convergence to a stable minimum. The final loss corresponds to $\mathcal{L}^* = (1 - \rho^2) \widehat{\text{Var}}(\phi)$, where $\rho > 0.90$ is the empirical correlation between payoffs and network outputs on the pilot set, implying a variance reduction factor exceeding $5\times$ relative to plain Monte Carlo.

Appendix B

Python Code Extracts

This appendix chapter reproduces the three critical algorithmic routines from `main.py` that implement Steps 3–5 of the pipeline. The extracts are presented in full, without abbreviation, to allow complete reproducibility of the numerical results reported in the main text. Syntax highlighting is restricted to bold keywords and italic comments, in keeping with the monochrome typographic standard of this document.

All code is written in Python 3.10+ and requires the following libraries: `numpy`, `scikit-learn`, `torch` (PyTorch), and `scipy`. The functions below are self-contained and callable once the upstream feature matrix `X_scaled` and pilot payoff vector `payoffs` have been constructed (Steps 1–2).

B.1 K-Means Clustering and Neyman Allocation (Step 3)

Mathematical background. Given a normalised feature matrix $\mathbf{X} = \{\mathbf{x}_i\}_{i=1}^N \subset \mathbb{R}^5$, K-Means++ solves the combinatorial optimisation problem:

$$\min_{\{C_k\}_{k=1}^K} \sum_{k=1}^K \sum_{\mathbf{x} \in C_k} \|\mathbf{x} - \boldsymbol{\mu}_k\|_2^2,$$

where $\boldsymbol{\mu}_k = |C_k|^{-1} \sum_{\mathbf{x} \in C_k} \mathbf{x}$ is the centroid of cluster k . Once the pilot paths are assigned to clusters, the Neyman optimal budget allocation concentrates simulation effort on the strata with the highest variance–weight product:

$$n_k^* = N \cdot \frac{p_k \sigma_k}{\sum_{j=1}^K p_j \sigma_j}, \quad p_k = \frac{|C_k|}{N}, \quad \sigma_k = \widehat{\text{Std}}(\phi \mid C_k).$$

The following listing implements both the clustering and the allocation calculation in a single coherent routine.

Listing B.1: K-Means++ clustering and Neyman optimal allocation (from `main.py`, Section 5).

```
1 from sklearn.cluster import KMeans
2 import numpy as np
3
4 def cluster_paths_kmeans(X_scaled, K, random_state=42):
5     """
6     K-Means++ clustering on normalised feature matrix.
```

```

7
8     Parameters
9     -----
10    X_scaled      : (N, d) float ndarray -- standardised
        features
11    K              : int                  -- number of
        clusters
12    random_state  : int
13
14    Returns
15    -----
16    labels       : (N,) int ndarray -- cluster assignment per
        path
17    centers      : (K, d) float ndarray -- centroid coordinates
18    km_model     : fitted KMeans object
19    """
20    km = KMeans(
21        n_clusters=K,
22        init="k-means++", gray gray#gray graysmartgray
        grayinitialisationgray gray(grayArthurgray gray&gray
        grayVassilvitskiigray,gray gray2007)
23        n_init=10, gray gray#gray gray10gray
        grayrestartsgray graytograd grayavoidgray graylocal
        gray grayminima
24        random_state=random_state
25    )
26    labels = km.fit_predict(X_scaled)
27    return labels, km.cluster_centers_, km
28
29
30 def neyman_allocation(payoffs, labels, K, N_total):
31     """
32     Compute Neyman-optimal simulation budget per cluster.
33
34     Formula:  $n_k^* = N * (p_k * \sigma_k) / \sum_j (p_j * \sigma_j)$ 
35
36     Parameters
37     -----
38     payoffs      : (N_pilot,) float ndarray -- pilot discounted
        payoffs

```

```

39     labels      : (N_pilot,) int ndarray    -- cluster
        assignments
40     K           : int                      -- number of clusters
41     N_total     : int                      -- total pricing
        budget
42
43     Returns
44     -----
45     allocation : (K,) int ndarray    -- paths allocated to
        each cluster
46     p_k        : (K,) float ndarray -- empirical cluster
        weights
47     sigma_k    : (K,) float ndarray -- within-cluster payoff
        std devs
48     """
49     N_pilot = len(payoffs)
50     p_k     = np.zeros(K)
51     sigma_k = np.zeros(K)
52
53     gray#gray grayStepgray gray3grayagray:gray graycomputegray
graywithingray-grayclustergray graystatistics
54     for k in range(K):
55         mask = labels == k
56         n_k  = mask.sum()
57         p_k[k] = n_k / N_pilot
58     gray#gray grayUsegray grayddofgray=1gray gray(
grayunbiasedgray)gray graystandardgray graydeviation
59         sigma_k[k] = payoffs[mask].std(ddof=1) if n_k >= 2
            else 0.0
60
61     gray#gray grayStepgray gray3graybgray:gray grayNeymangray
grayweights
62         denom = (p_k * sigma_k).sum()
63         weights = (p_k * sigma_k / denom) if denom > 0 else p_k
64
65     gray#gray grayStepgray gray3graycgray:gray grayroundgray
graytogray grayintegergray graycountsgray,gray graythengray
graycorrectgray grayroundinggray grayerror
66     allocation = np.maximum(
67         np.round(weights * N_total).astype(int), 1
68 )

```

```

69 gray      gray#gray grayEnsuregray grayallocationgray graysumsgray
      graytogroup grayexactlygray grayN_total
70      allocation[np.argmax(weights)] += N_total - allocation.
      sum()
71
72      return allocation, p_k, sigma_k

```

B.2 Local Random Forest Control Variate (Step 4)

Mathematical background. For each cluster k , a Random Forest regressor $\hat{f}_k : \mathbb{R}^5 \rightarrow \mathbb{R}$ is fitted on the pilot paths belonging to C_k . The locally corrected payoff estimator for cluster k is:

$$\tilde{\phi}_i^{(k)} = \phi_i - c_k^* (\hat{f}_k(\mathbf{x}_i) - \mu_{Y,k}), \quad c_k^* = \frac{\widehat{\text{Cov}}(\phi, \hat{f}_k)}{\widehat{\text{Var}}(\hat{f}_k)}, \quad \mu_{Y,k} = \mathbb{E}[\hat{f}_k(X) \mid X \in C_k],$$

where c_k^* is the OLS-optimal scalar coefficient and $\mu_{Y,k}$ ensures that the control variate correction is mean-zero, preserving the unbiasedness of the estimator. The global stratified price estimate combines the within-cluster corrected means weighted by the empirical cluster proportions p_k .

Listing B.2: Local Random Forest control variate training and application (from `main.py`, Section 7).

```

1 from sklearn.ensemble import RandomForestRegressor
2 import numpy as np
3
4 def rf_control_variate(
5     X_pilot, payoffs_pilot, labels_pilot,
6     X_pricing, labels_pricing, p_k, K,
7     n_estimators=200, max_depth=None, random_state=42
8 ):
9     """
10    Fit one Random Forest per cluster; use predictions as
11    local control variates to reduce within-cluster variance.
12
13    Parameters
14    -----
15    X_pilot          : (N_pilot, d)    normalised pilot
        features
16    payoffs_pilot    : (N_pilot,)      discounted pilot payoffs
17    labels_pilot     : (N_pilot,)      cluster labels (pilot)

```

```

18     X_pricing      : (N_price, d)    normalised pricing
        features
19     labels_pricing : (N_price,)      cluster labels (pricing)
20     p_k            : (K,)            empirical cluster
        weights
21     K              : int              number of clusters
22
23     Returns
24     -----
25     estimate        : float          -- corrected price estimate
26     residual_var_ratio: float        -- Var(corrected) / Var(raw)
27     """
28     forests         = {} gray    gray#gray grayonegray grayRFgray
        graymodelgray graypergray graycluster
29     ctrl_means      = {} gray    gray#gray grayEgray[grayf_kgray(grayX
        gray)gray gray/gray grayXgray grayingray grayC_kgray]gray
        gray gray(grayforgray graymeangray-grayzerogray
        graycorrectiongray)
30
31 gray    gray#gray gray    gray grayTraininggray grayphasegray:
        gray grayfitgray grayonegray grayRFgray graypergray
        grayclustergray
        gray
32     for k in range(K):
33         mask = labels_pilot == k
34         if mask.sum() < 10: gray    gray#gray
            grayinsufficientgray graydatagray:gray grayskip
35             forests[k]      = None
36             ctrl_means[k]   = 0.0
37             continue
38
39         rf = RandomForestRegressor(
40             n_estimators=n_estimators,
41             max_depth=max_depth,
42             min_samples_leaf=5, gray    gray#gray grayprevents
                gray grayoverfittinggray grayongray graysmallgray
                grayleaves
43             n_jobs=-1,
44             random_state=random_state
45         )
46         rf.fit(X_pilot[mask], payoffs_pilot[mask])

```

```

47         forests[k]      = rf
48 gray         gray#gray grayControlgray grayvariategray graymeangray
              gray(grayestimatedgray grayongray graypilotgray
              graytraininggray graydatagray)
49         ctrl_means[k] = rf.predict(X_pilot[mask]).mean()
50
51 gray         gray#gray gray         gray grayPricinggray grayphasesgray:gray
              grayapplygray graylocalgray grayOLSgray graycorrectiongray
              gray
52         corrected, raw = [], []
53
54         for k in range(K):
55             mask_tr = labels_pilot == k
56             mask_pr = labels_pricing == k
57             n_tr     = mask_tr.sum()
58             n_pr     = mask_pr.sum()
59
60             if n_pr == 0:
61                 continue
62
63 gray         gray#gray grayAligngray graypilotgray graysamplegray
              graytogray graypricinggray grayclustergray graysize
64             phi_pr = (payoffs_pilot[mask_tr][:n_pr]
65                      if n_tr >= n_pr
66                      else payoffs_pilot[mask_tr])
67             if len(phi_pr) == 0:
68                 continue
69
70             raw.extend(phi_pr.tolist())
71
72             if forests[k] is None: gray         gray#gray grayfallback
              gray:gray graynogray graycorrection
73                 corrected.extend(phi_pr.tolist())
74                 continue
75
76             X_pr     = X_pricing[mask_pr][:len(phi_pr)]
77             Y_pr     = forests[k].predict(X_pr)
78             mu_Y_k   = ctrl_means[k]
79
80 gray         gray#gray grayOLSgray grayoptimalgray graycoefficient
              gray:gray graycgray*gray gray=gray grayCougray(grayphi

```

```

81         gray,gray grayYgray)gray gray/gray grayVargray(grayYgray)
82     cov_kk = np.cov(phi_pr, Y_pr)[0, 1]
83     var_Y   = Y_pr.var(ddof=1)
84     c_k      = cov_kk / var_Y if var_Y > 1e-10 else 0.0
85 gray       gray#gray grayApplygray graymeangray-grayzerogray
86             graycontrolgray grayvariategray graycorrection
87     corrected.extend(
88         (phi_pr - c_k * (Y_pr - mu_Y_k)).tolist()
89     )
90     corrected = np.array(corrected)
91     raw       = np.array(raw)
92     raw_var   = raw.var(ddof=1) if raw.var(ddof=1) > 0 else
93         1.0
94     return corrected.mean(), corrected.var(ddof=1) / raw_var

```

B.3 Neural Network Variance-Loss Optimiser (Step 5)

Mathematical background. The global neural network control variate solves:

$$\min_{\theta \in \Theta, c \in \mathbb{R}} \mathcal{L}(\theta, c) = \mathbb{E} \left[\left(\phi - c g_{\theta}(\mathbf{x}) \right)^2 \right],$$

which equals $\text{Var}(\phi) - 2c \text{Cov}(\phi, g_{\theta}) + c^2 \text{Var}(g_{\theta})$. The minimiser over c at fixed θ is the OLS coefficient $c^* = \text{Cov}(\phi, g_{\theta}) / \text{Var}(g_{\theta})$, and at the global optimum $\mathcal{L}^* = (1 - \rho^2) \text{Var}(\phi)$ where $\rho = \text{Corr}(\phi, g_{\theta})$. The alternating update — gradient step in θ , closed-form update of c — ensures monotone decrease of the loss. The network architecture is:

$$g_{\theta} : \mathbb{R}^5 \xrightarrow{\text{Linear}_{64}} \text{ReLU} \xrightarrow{\text{Linear}_{64}} \text{ReLU} \xrightarrow{\text{Linear}_{64}} \text{ReLU} \xrightarrow{\text{Linear}_1} \mathbb{R}.$$

Weights are initialised with He (Kaiming) normal initialisation for ReLU activations.

Listing B.3: Neural network architecture and custom variance-loss training routine (from `main.py`, Section 8).

```

1 import torch
2 import torch.nn as nn
3 import torch.optim as optim
4 import numpy as np
5

```



```

6
7 class ControlVariateNet(nn.Module):
8     """
9     Feedforward network  $g_{\theta} : \mathbb{R}^d \rightarrow \mathbb{R}$ .
10
11     Architecture : Input  $\rightarrow$  [Linear(64)  $\rightarrow$  ReLU] x 3  $\rightarrow$ 
12                     Linear(1)
13     Initialisation: He (Kaiming) normal for ReLU layers.
14     """
15     def __init__(self, d_in: int):
16         super().__init__()
17         self.net = nn.Sequential(
18             nn.Linear(d_in, 64), nn.ReLU(),
19             nn.Linear(64, 64), nn.ReLU(),
20             nn.Linear(64, 64), nn.ReLU(),
21             nn.Linear(64, 1),
22         )
23
24         #gray#gray grayHegray grayinitialisationgray:gray
25         #graypreservesgray grayvariancegray grayacrossgray
26         #grayReLUgray graylayers
27
28         for m in self.modules():
29             if isinstance(m, nn.Linear):
30                 nn.init.kaiming_normal_(m.weight,
31                                         nonlinearity="relu")
32                 nn.init.zeros_(m.bias)
33
34     def forward(self, x):
35         return self.net(x).squeeze(-1)
36
37 def train_nn_control_variate(
38     X_train, phi_train,
39     n_epochs=200, batch_size=256, lr=1e-3,
40     weight_decay=1e-4, dropout_p=0.1, device="cpu",
41     verbose=False, record_loss=False,
42 ):
43     """
44     Train ControlVariateNet with the custom variance loss
45      $L(\theta, c) = E[(\phi - c * g_{\theta}(x))^2]$ .
46
47     The scalar  $c$  is updated in closed form after each epoch:

```

```

43         c* = Cov(phi, g_theta) / Var(g_theta)
44
45     Parameters
46     -----
47     X_train      : (N, d) float ndarray  -- normalised
         features
48     phi_train    : (N,)   float ndarray  -- discounted
         payoffs
49     n_epochs     : int      training epochs
50     batch_size   : int      mini-batch size (SGD)
51     lr           : float    Adam learning rate
52     weight_decay : float    L2 regularisation coefficient
53     dropout_p    : float    hidden-layer dropout probability
54     device       : str      'cpu' or 'cuda'
55     record_loss  : bool     if True, return per-epoch loss
         history
56
57     Returns
58     -----
59     net          : trained ControlVariateNet
60     c_star       : float     -- optimal scalar coefficient
61     mu_g         : float     -- E[g_theta(x)] on training set
62     loss_hist    : list      -- per-epoch losses (if record_loss=
         True)
63     """
64     X_t = torch.tensor(X_train, dtype=torch.float32).to(
         device)
65     phi_t = torch.tensor(phi_train, dtype=torch.float32).to(
         device)
66     N = X_train.shape[0]
67
68     net = ControlVariateNet(X_train.shape[1]).to(device
        )
69     optimiser = optim.Adam(
70         net.parameters(), lr=lr, weight_decay=weight_decay
71     )
72     c_star = 1.0 gray          gray#gray grayinitialisegray
         graycoefficient
73     loss_hist = []
74
75     for epoch in range(n_epochs):

```

```

76         net.train()
77         perm          = torch.randperm(N)
78         epoch_loss    = 0.0
79         n_batches     = 0
80
81         for start in range(0, N, batch_size):
82             idx      = perm[start: start + batch_size]
83             x_b      = X_t[idx]
84             phi_b     = phi_t[idx]
85
86             optimiser.zero_grad()
87             g_b = net(x_b)
88
89 gray             gray#gray grayCustomgray grayvariancegray grayloss
90                 gray:gray grayVargray(grayphigray gray-gray grayc
91                 gray gray*gray grayg_thetagrays)
92             loss = ((phi_b - c_star * g_b) ** 2).mean()
93             loss.backward() gray gray#gray graybackpropagate
94                 gray graythroughgray grayg_thetagrays grayonly
95             optimiser.step()
96             epoch_loss += loss.item()
97             n_batches += 1
98
99 gray             gray#gray grayClosedgray-grayformgray grayupdategray
100                 grayofgray graycgray grayaftergray grayeachgray grayfull
101                 gray grayepoch
102             net.eval()
103             with torch.no_grad():
104                 g_all      = net(X_t)
105                 phi_mean   = phi_t.mean()
106                 g_mean     = g_all.mean()
107 gray             gray#gray graycgray*gray gray=gray grayCovgray(
108                 grayphigray,gray grayggray)gray gray/gray grayVar
109                 gray(grayggray)
110             num = ((phi_t - phi_mean) * (g_all - g_mean)).
111                 mean().item()
112             den = ((g_all - g_mean) ** 2).mean().item()
113             if abs(den) > 1e-10:
114                 c_star = num / den
115
116         avg_loss = epoch_loss / max(n_batches, 1)

```

```

109         loss_hist.append(avg_loss)
110
111     if verbose and (epoch + 1) % 50 == 0:
112         print(
113             f"    Epoch {epoch+1:3d}/{n_epochs}    "
114             f"loss={avg_loss:.6f}    c*={c_star:.4f}"
115         )
116
117 gray     gray#gray grayFinalgray graymeangray grayofgray grayg_theta
gray gray(grayforgray grayunbiasedgray graypricegray
graygraycorrectiongray)
118 net.eval()
119 with torch.no_grad():
120     mu_g = net(X_t).mean().item()
121
122     return (net, c_star, mu_g, loss_hist) if record_loss \
123           else (net, c_star, mu_g)

```

Appendix C

Supplementary Parameter Tables

Table C.1 summarises the model and simulation parameters used throughout the numerical study. Table C.2 collects the hyperparameters of the neural network control variate.

Table C.1: **Model and simulation parameters.** Parameters are held fixed across all experiments unless otherwise noted in the main text.

Symbol	Description	Value	Unit
S_0	Initial asset price	100	\$
K	Strike price	100	\$
r	Risk-free rate	0.05	ann. rate
σ	GBM volatility	0.20	ann. vol
T	Maturity	1.0	years
M	Time steps	252	steps (daily)
N_{pilot}	Pilot paths	20 000	paths
N_{price}	Pricing paths	20 000	paths
K_{clust}	Number of clusters	8	—
κ	Heston mean-reversion speed	2.0	—
θ	Heston long-run variance	0.04	—
ξ	Heston vol-of-vol	0.30	—
ρ	Heston correlation	−0.70	—
V_0	Heston initial variance	0.04	—

Table C.2: **Neural network hyperparameters.** The architecture and training configuration are held fixed across all option types and model dynamics.

Hyperparameter	Description	Value
Architecture	Hidden layers $\times$ units	3×64
Activation	Hidden activation	ReLU
Initialisation	Weight initialisation	He (Kaiming) normal
Optimiser	Gradient method	Adam
Learning rate	α	10^{-3}
Weight decay	L2 regularisation λ	10^{-4}
Dropout	Hidden-layer dropout p	0.10
Batch size	Mini-batch size	256
Epochs	Training epochs	200
Pilot size	Pre-training paths N_0	50 000

Bibliography

- [1] Andersen, L. (2008). *Simple and efficient simulation of the Heston stochastic volatility model*. Journal of Computational Finance, 11(3), 1–42.
- [2] Arthur, D. & Vassilvitskii, S. (2007). *k-means++: The advantages of careful seeding*. In Proceedings of the 18th Annual ACM-SIAM Symposium on Discrete Algorithms (SODA), 1027–1035.
- [3] Black, F. & Scholes, M. (1973). *The pricing of options and corporate liabilities*. Journal of Political Economy, 81(3), 637–654.
- [4] Breiman, L., Friedman, J. H., Olshen, R. A., & Stone, C. J. (1984). *Classification and Regression Trees*. Wadsworth, Belmont, CA.
- [5] Breiman, L. (1996). *Bagging predictors*. Machine Learning, 24(2), 123–140.
- [6] Breiman, L. (2001). *Random forests*. Machine Learning, 45(1), 5–32.
- [7] Cybenko, G. (1989). *Approximation by superpositions of a sigmoidal function*. Mathematics of Control, Signals, and Systems, 2(4), 303–314.
- [8] Delbaen, F. & Schachermayer, W. (1994). *A general version of the fundamental theorem of asset pricing*. Mathematische Annalen, 300(1), 463–520.
- [9] Glasserman, P. (2004). *Monte Carlo Methods in Financial Engineering*. Springer, New York.
- [10] Harrison, J. M. & Kreps, D. M. (1979). *Martingales and arbitrage in multiperiod securities markets*. Journal of Economic Theory, 20(3), 381–408.
- [11] Heston, S. L. (1993). *A closed-form solution for options with stochastic volatility with applications to bond and currency options*. Review of Financial Studies, 6(2), 327–343.
- [12] He, K., Zhang, X., Ren, S., & Sun, J. (2015). *Delving deep into rectifiers: Surpassing human-level performance on ImageNet classification*. In Proceedings of ICCV, 1026–1034.
- [13] Hornik, K. (1991). *Approximation capabilities of multilayer feedforward networks*. Neural Networks, 4(2), 251–257.
- [14] Kemna, A. G. Z. & Vorst, A. C. F. (1990). *A pricing method for options based on average asset values*. Journal of Banking and Finance, 14(1), 113–129.

- [15] Kingma, D. P. & Ba, J. (2015). *Adam: A method for stochastic optimization*. In International Conference on Learning Representations (ICLR).
- [16] Longstaff, F. A. & Schwartz, E. S. (2001). *Valuing American options by simulation: A simple least-squares approach*. Review of Financial Studies, 14(1), 113–147.
- [17] Lundberg, S. M. & Lee, S.-I. (2017). *A unified approach to interpreting model predictions*. Advances in Neural Information Processing Systems (NeurIPS), 30.
- [18] Merton, R. C. (1973). *Theory of rational option pricing*. Bell Journal of Economics and Management Science, 4(1), 141–183.
- [19] Rumelhart, D. E., Hinton, G. E., & Williams, R. J. (1986). *Learning representations by back-propagating errors*. Nature, 323(6088), 533–536.
- [20] Scornet, E., Biau, G., & Vert, J.-P. (2015). *Consistency of random forests*. Annals of Statistics, 43(4), 1716–1741.
- [21] Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I., & Salakhutdinov, R. (2014). *Dropout: A simple way to prevent neural networks from overfitting*. Journal of Machine Learning Research, 15(56), 1929–1958.